Laboratory on processor design, guide
Course Computer Technology

The student designs a circuit that represents a working processor and simulates it using binary instructions (in hexadecimal). The basic components are given in an implemented way in Logisim in the file TG00RISCV.zip.

The TG00RISCV procesador has:

- Words of 32 bits.
- Register file with 32 registers of 32 bits. Register x0 has a value of 0.
- An ALU of 32 bits with:
  - C2 add and subtract on integer operands
  - Logic operations AND, OR and NOR.
  - Set on less than.
  - Z flag to indicate (value 1) the ALU result is 0.
- Instruction memory and Data memory:
  - Address and data bus of 32 bits
  - Addressing on byte level

Implemented components are the ALU, register file, instruction and data memory, immediate extension module and ALU control. See details in Appendix I. Appendix II provides the instructions to be implemented in TG00RISCV.

We are going to implement the circuit step by step starting with the fetching and the jump instruction. In each stage, we load a program in instmem.txt and generate a simulation table to check it is working OK via the tab *..simulate* → *logging* → *selection.* You will notice that the use of labels and probes is very useful. Remember that for RISCV, we increase the PC by 4.

Exercise 1 jump and halt

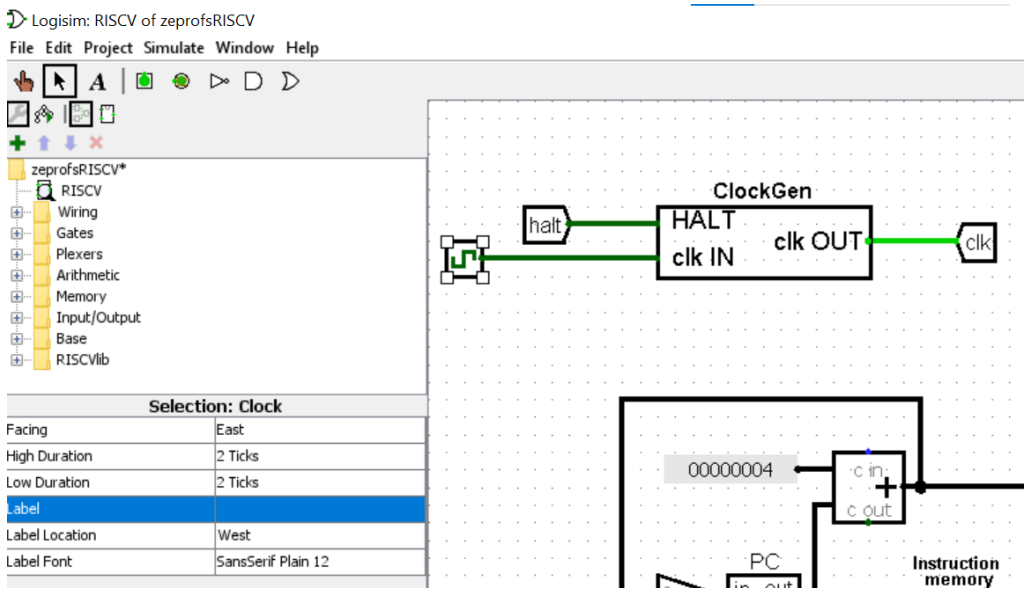
With the instruction memory, we can extend the updating of the PC with the Jump instruction and create an explicit halt instruction (not a RISCV instruction). The given instruction memory loads the content of file **instmem.txt** on reset. It has a header "**v2.0 raw**" after which we can put line wise the hexadecimal code.

Pos.(hex)	Instructions	Binary code in hex
0	nop	00000000
4	nop	00000004
8	jal x1, five	00c000ef
C	three: nop	0000000C
10	jal x1, seven	00c000ef
14	five: nop	00000014
18	jal x1, three	ff5ff0ef
1C	seven: nop	0000001C
20	halt	ffffffff

a. Fetching

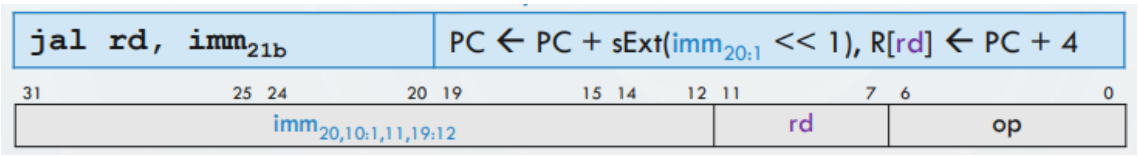
Notice that even this small circuit requires a minimum of control; we should know the instruction is a jump and use a multiplexor to decide the next PC is the jump address or **PC+4**. The only signal in your control unit is jump.

In this part of the course, we also focus on the delays in the components, which have been implemented to simulate the complete system. Therefore, we have to set the cycle time of the clock as indicated in an exotic language in the figure. In the picture, the clock is set as 2 ticks for high duration and two ticks for low duration. Given that the delay of reading the memory is 4 ticks, we require the clock cycle to be at least 4 ticks to be able to read the instruction during the cycle.



b. Implementation jal

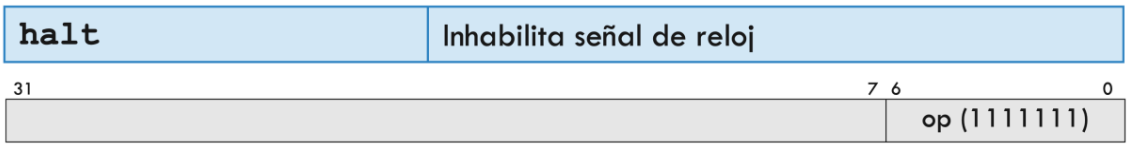
The JAL instruction (**op 1101111**) loads in the PC a new address which adds to the PC the bit extension of the immediate (imm) field. the imm field. For the bit extension, you can use the extension circuit of Appendix I.



Type-J

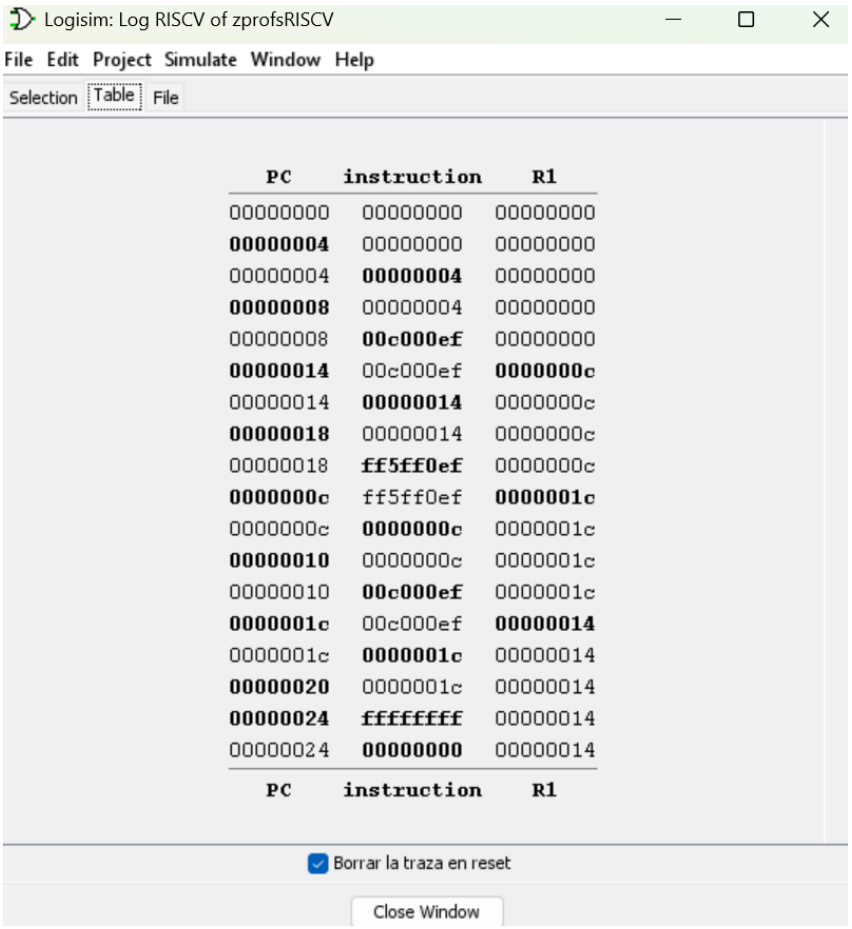
c. Implementation of halt

This is not a RISCV instruction, but we use it to stop the clock as we did before now using the provided circuit ClockGen.



Basically, we activate a signal halt on instruction halt and the circuit stops.

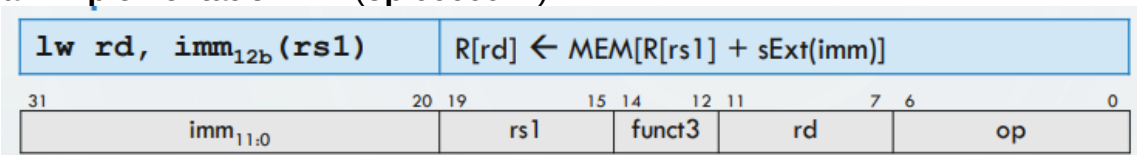
Now having implemented fetching, jal an halt, we can simulate the first program (given in the table) and create a simulation table (trace) which shows **PC, instruction.** This is the first circuit to hand in in **GUAC**. If you think the trace is correct, upload into GUAC with a file **teamname**RISCV.zip.



Exercise 2, LW and SW

We extend our earlier (JAL and HALT) circuit now with more hardware elements, like the data memory.

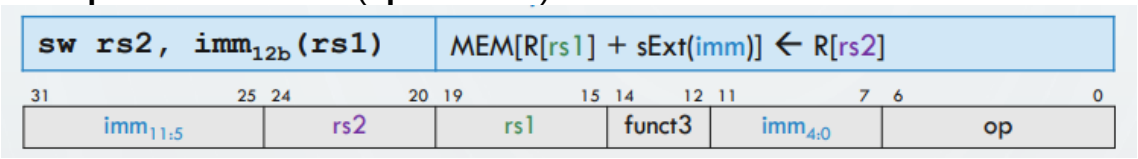
a. Implementation LW (op 0000011)



Type-I

The instruction takes the content of a memory position and puts it into register rd. The memory address is typically calculated (base register) taking the content of register rs1 and adding a bit extended value for the immediate imm. For the bit extension, we use again the provided circuit of Appendix I. I would say, have a look at the (very many) slides to have the scheme. Don't forget to connect the clock with register file and data memory.

b. Implementation SW (op 0100011)



Type-S

In store word, we go the other way around, but we have the same operands. Take care to adapt the clock cycle time, such that the whole instruction can complete the path with the delays of memory, ALU, and register file.

- **Instruction memory: 4** ticks)
- **Data memory: 4** ticks for as well reading as writing!
- **ALU: 4** ticks
- **Register file: 2** ticks for as well reading as writing!

There we go. Get it working and show it loading the following program and data.

Instruction memory

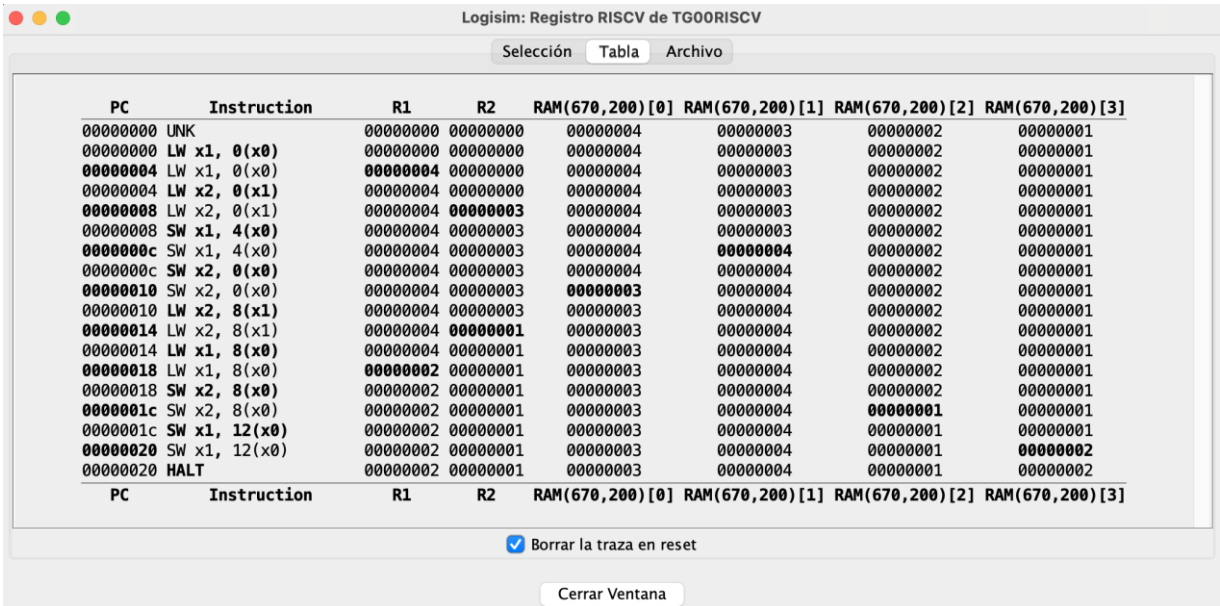
Pos.(hex)	Instructions	Content instmem.txt
0	lw x1, 0(x0)	00002083
4	lw x2, 0(x1)	0000a103
8	sw x1, 4(x0)	00102223
C	sw x2, 0(x0)	00202023
10	lw x2, 8(x1)	0080a103
14	lw x1, 8(x0)	00802083
18	sw x2, 8(x0)	00202423
1C	sw x1, 0xC(x0)	00102623
20	halt	ffffffff

Data memory

Pos.(hex)	Content datamem.txt
0	00000004
4	00000003
8	00000002
C	00000001
10	00000000
14	
18	
1C	
20	

How to show it is correct? Create a trace showing the hexadecimal content of **PC, R1, R2** and the first **4 data** memory positions. It is handy to also provide the instruction. It is tuned in such a way that if you use Base “32” it shows a de-codification of the instruction (if that instruction exists:-)

This is what my colleague got out:

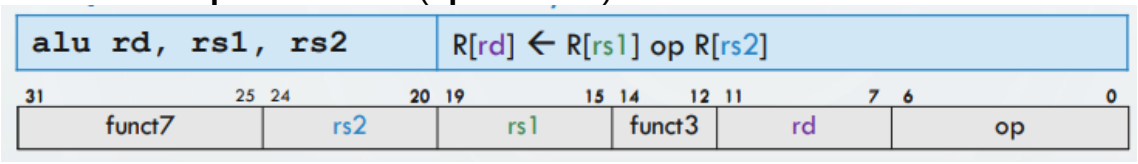


PC	Instruction	R1	R2	RAM(670,200)[0]	RAM(670,200)[1]	RAM(670,200)[2]	RAM(670,200)[3]
00000000	UNK	00000000	00000000	00000004	00000003	00000002	00000001
00000000	**LW x1, 0(x0)**	00000000	00000000	00000004	00000003	00000002	00000001
00000004	LW x1, 0(x0)	**00000004**	00000000	00000004	00000003	00000002	00000001
00000004	**LW x2, 0(x1)**	00000004	00000000	00000004	00000003	00000002	00000001
00000008	LW x2, 0(x1)	00000004	**00000003**	00000004	00000003	00000002	00000001
00000008	**SW x1, 4(x0)**	00000004	00000003	00000004	00000003	00000002	00000001
0000000c	SW x1, 4(x0)	00000004	00000003	00000004	**00000004**	00000002	00000001
0000000c	**SW x2, 0(x0)**	00000004	00000003	00000004	00000004	00000002	00000001
00000010	SW x2, 0(x0)	00000004	00000003	**00000003**	00000004	00000002	00000001
00000010	**LW x2, 8(x1)**	00000004	00000003	00000003	00000004	00000002	00000001
00000014	LW x2, 8(x1)	00000004	**00000001**	00000003	00000004	00000002	00000001
00000014	**LW x1, 8(x0)**	00000004	00000001	00000003	00000004	00000002	00000001
00000018	LW x1, 8(x0)	**00000002**	00000001	00000003	00000004	00000002	00000001
00000018	**SW x2, 8(x0)**	00000002	00000001	00000003	00000004	00000002	00000001
0000001c	SW x2, 8(x0)	00000002	00000001	00000003	00000004	**00000001**	00000001
0000001c	**SW x1, 12(x0)**	00000002	00000001	00000003	00000004	00000001	00000001
00000020	SW x1, 12(x0)	00000002	00000001	00000003	00000004	00000001	**00000002**
00000020	**HALT**	00000002	00000001	00000003	00000004	00000001	00000002
PC	Instruction	R1	R2	RAM(670,200)[0]	RAM(670,200)[1]	RAM(670,200)[2]	RAM(670,200)[3]

Exercise 3, ALU and ALUI instructions

We are going to add hardware and control to have ALU and ALUI working besides the earlier implemented JAL, HALT, LW and SW.

a. ALU implementation (op 0110011)

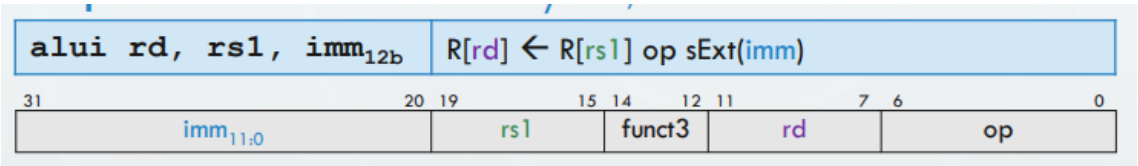


Type-R

ALU	opALU	funct7	funct3
ADD	+	0000000	000
SUB	-	0100000	000
AND	&	0000000	111
OR	|	0000000	110
SLT	slt	0000000	010

The operands are the destination register rd and the source registers rs1 and rs2. The operation to be carried out is provided by the funct7 and funct3 fields. The ALU control is fed with these fields and the indication it is type R.

b. ALUI implementation (op 0010011)



Type-I

ALU	opALU	funct3
ADDI	+	000
ANDI	&	111
ORI	|	110
SLTI	slt	010

The operands are a destination register rd and a source register rs1 and immediate extended to 32 bits. In the provided bit extendor we can select type-I as instruction format. The ALU control will be provided field funct3 and the indication we are working with type I.

Now get it working and show with the fillowing input that this might be the case.

Instruction memory

Pos.(hex)	Instruction	Content instmem.txt
0	lw x1, 4(x0)	00402083
4	lw x2, 4(x1)	0040a103
8	add x3, x2, x1	001101b3
C	sub x3, x3, x2	402181b3
10	addi, x3, x2, -500	e0c10193
14	ori x2, x2, 0x7a0	7a016113
18	andi x3, x3, 0x7ff	7ff1f193
1C	and x1, x2, x3	003170b3
20	or x1, x2, x3	003160b3
24	slt x3, x1, x2	0020a1b3
28	slt x3, x2, x1	001121b3
2C	slti x3, x1, -0x7ff	8010a193
30	slti x3, x1, 0x7ff	7ff0a193
34	sw x1, 0(x0)	00102023
38	halt	ffffffff

Data memory

Pos.(hex)	Content datamem.txt
0	3
4	-4
8	0

In the Logisim trace, show the content of **PC, R1, R2, R3** and the first data memory position in hexadecimal. You'd better add the instruction to the table.

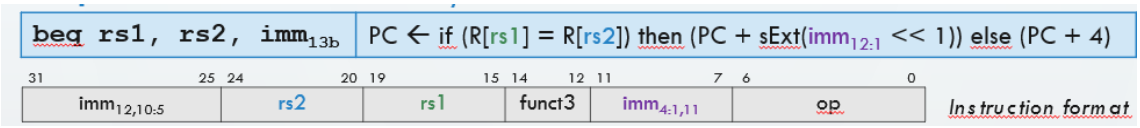
Please, don't be alarmed by the "strange" representation. In the end, we are talking about C2 integer numbers represented in hexadecimal. So fffffffc is not a football club, but represents the number -4.

My colleague got out the following table.

	PC	R1	R2	R3	RAM[0]
lw x1, 4(x0)	00000000	00000000	00000000	00000000	00000003
lw x2, 4(x1)	00000004	**fffffffc**	00000000	00000000	00000003
add x3, x2, x1	00000008	fffffffc	**00000003**	00000000	00000003
sub x3, x3, x2	0000000c	fffffffc	00000003	**ffffffff**	00000003
addi, x3, x2, -500	00000010	fffffffc	00000003	**fffffffc**	00000003
ori x2, x2, 0x7a0	00000014	fffffffc	00000003	**fffffe0f**	00000003
andi x3, x3, 0x7ff	00000018	fffffffc	**000007a3**	fffffe0f	00000003
and x1, x2, x3	0000001c	fffffffc	000007a3	**0000060f**	00000003
or x1, x2, x3	00000020	**00000603**	000007a3	0000060f	00000003
slt x3, x1, x2	00000024	**000007af**	000007a3	0000060f	00000003
slt x3, x2, x1	00000028	000007af	000007a3	**00000000**	00000003
slti x3, x1, -0x7ff	0000002c	000007af	000007a3	**00000001**	00000003
slti x3, x1, 0x7ff	00000030	000007af	000007a3	**00000000**	00000003
sw x1, 0(x0)	00000034	000007af	000007a3	**00000001**	00000003
halt	00000038	000007af	000007a3	00000001	**000007af**

Exercise 4, finally getting BEQ (op 1100011)

BEQ compares the content of the registers rs1 and rs2. If they are not equal, we go for PC+4 and otherwise we add to PC the bite extended immediate imm.



Type-B

For the bit extension, we can use the provided bit extender specifying we are dealing with an instruction type B. Take care. To check the equality of the registers, we should use the ALU subtracting the contents. If the result is zero, we use the Zero signal output of the ALU.

Now we can check your implementation with the following data which simulates a loop program over an array of data to find its sum.

Instruction memory

Pos. (hex)	Instruction	Content instmem.txt
0	lw x1, 0x14(x0)	01402083
4	addi x2, x0, 4	00400113
8	sub x3, x3, x3	403181b3
C	add x4, x0, x0	00000233
10	lo: beq x4, x1, sl	00120c63
14	lw x5, 0(x2)	00012283
18	add x3, x3, x5	005181b3
1C	addi x2, x2, 4	00410113
20	addi x4, x4, 1	00120213
24	jal x0, lo	fedff06f
28	sl: sw x3, 0(x0)	00302023
2C	halt	ffffffff

Data memory

Dir. (hex)	Content datamem.txt	
0	00000000	Result
4	fffffffe	Vector
8	00000004	
C	fffffffa	
10	00000008	
14	00000004	numel

Create a trace in Logisim which shows the content of **PC, R1, R2, R3, R4, R5** and the first data memory position in hexadecimal.

Compare with the table my colleague got out.

		PC	R1	R2	R3	R4	R5	RAM[0]
	lw x1, 0x14(x0)	00000000	00000000	00000000	00000000	00000000	00000000	00000000
	addi x2, x0, 4	00000004	**00000004**	00000000	00000000	00000000	00000000	00000000
	sub x3, x3, x3	00000008	00000004	**00000004**	00000000	00000000	00000000	00000000
	add x4, x0, x0	0000000c	00000004	00000004	**00000000**	00000000	00000000	00000000
lo:	beq x4, x1, sl	00000010	00000004	00000004	00000000	**00000000**	00000000	00000000
	lw x5, 0(x2)	00000014	00000004	00000004	00000000	00000000	00000000	00000000
	add x3, x3, x5	00000018	00000004	00000004	00000000	00000000	**fffffffe**	00000000
	addi x2, x2, 4	0000001c	00000004	00000004	**fffffffe**	00000000	fffffffe	00000000
	addi x4, x4, 1	00000020	00000004	**00000008**	fffffffe	00000000	fffffffe	00000000
	j lo	00000024	00000004	00000008	fffffffe	**00000001**	fffffffe	00000000
lo:	beq x4, x1, sl	**00000010**	00000004	00000008	fffffffe	00000001	fffffffe	00000000
	lw x5, 0(x2)	00000014	00000004	00000008	fffffffe	00000001	fffffffe	00000000
	add x3, x3, x5	00000018	00000004	00000008	fffffffe	00000001	**00000004**	00000000
	addi x2, x2, 4	0000001c	00000004	00000008	**00000002**	00000001	00000004	00000000
	addi x4, x4, 1	00000020	00000004	**0000000c**	00000002	00000001	00000004	00000000
	j lo	00000024	00000004	0000000c	00000002	**00000002**	00000004	00000000
lo:	beq x4, x1, sl	**00000010**	00000004	0000000c	00000002	00000002	00000004	00000000
	lw x5, 0(x2)	00000014	00000004	0000000c	00000002	00000002	00000004	00000000
	add x3, x3, x5	00000018	00000004	0000000c	00000002	00000002	**fffffffa**	00000000
	addi x2, x2, 4	0000001c	00000004	0000000c	**fffffffc**	00000002	fffffffa	00000000
	addi x4, x4, 1	00000020	00000004	**00000010**	fffffffc	00000002	fffffffa	00000000
	j lo	00000024	00000004	00000010	fffffffc	**00000003**	fffffffa	00000000
lo:	beq x4, x1, sl	**00000010**	00000004	00000010	fffffffc	00000003	fffffffa	00000000
	lw x5, 0(x2)	00000014	00000004	00000010	fffffffc	00000003	fffffffa	00000000
	add x3, x3, x5	00000018	00000004	00000010	fffffffc	00000003	**00000008**	00000000
	addi x2, x2, 4	0000001c	00000004	00000010	**00000004**	00000003	00000008	00000000
	addi x4, x4, 1	00000020	00000004	**00000014**	00000004	00000003	00000008	00000000
	j lo	00000024	00000004	00000014	00000004	**00000004**	00000008	00000000
lo:	beq x4, x1, sl	**00000010**	00000004	00000014	00000004	00000004	00000008	00000000
sl:	sw x3, 0(x0)	**00000028**	00000004	00000014	00000004	00000004	00000008	00000000
	halt	0000002c	00000004	00000014	00000004	00000004	00000008	**00000004**

Exercise 5, programming our processor

We are now going to use the code of GUAC assembly programming Session 8, which determines the maximum of a given array.

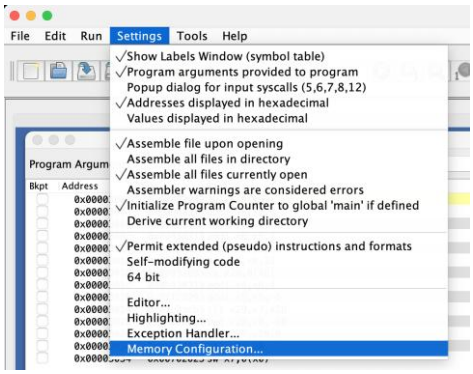
```
.data
res:     .word 0
numel:     .word 5
vector: .word 2, -3, 6, 5, -4

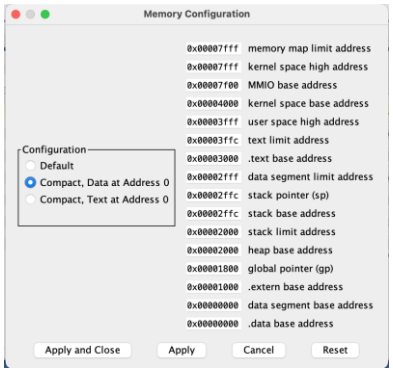
.text
main: lw x1, 4(zero)      # numel is in position 4 of the memory
      addi x2, zero, 8    # x2 = 8 position array
      lw x3, 0(x2)        # Initialize max with first element
      addi x2, x2, 4      # increment array pointer
      addi x1, x1, -1     # decrement counter
loop: beq x1, zero, exit  # If x1 == 0, end of loop
      lw x4, 0(x2)        # x4 = array[i]
      addi x2, x2, 4      # Increment array pointer
      addi x1, x1, -1     # Decrement counter
      slt x5, x3, x4      # if x3 < x4, x5 = 1
      beq x5, zero, loop  # If x5 == 0, next iteration
      addi x3, x4, 0      # If x5 == 1, put x4 as new maximum
      j loop              # Back to loop
exit: sw x3, 0(zero)      # Save máximum in "res" (0)
#     halt                # Code by hand; is not a RISCV instruction
```

To get the code out of RARS, we can use the following stages

1. **Get the instruction memory to start in 0**

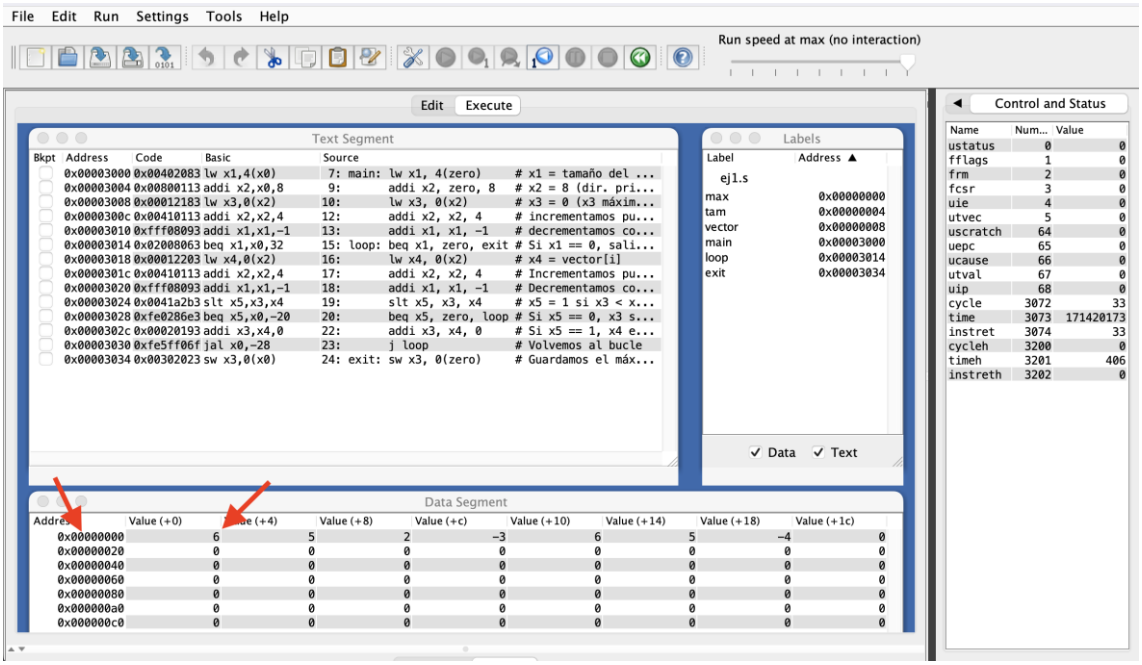
In *Settings*" → *Memory Configuration*… → *Compact, Data at Address 0*".





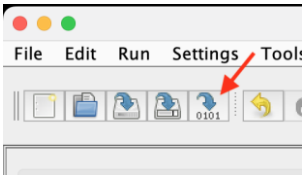
2. **Copy the code in RARS**

Take care of strange symbols (copy from pdf) and the order of the labels; they should be the same to coincide with the code.

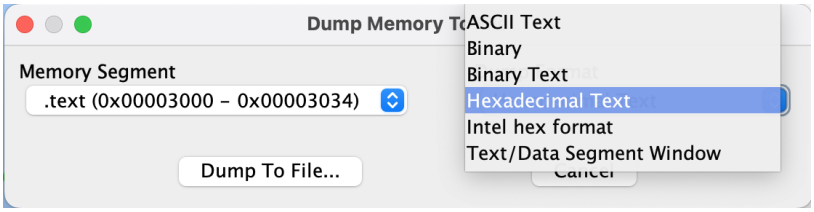


3. **Export hexadecimal code for instructions and data**

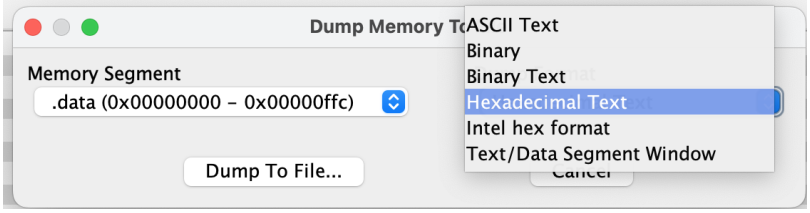
Select Dump memory



Export hexadecimal text to *instmem.txt*



Do the same with the data into *datamem.txt* starting the data in position 0.



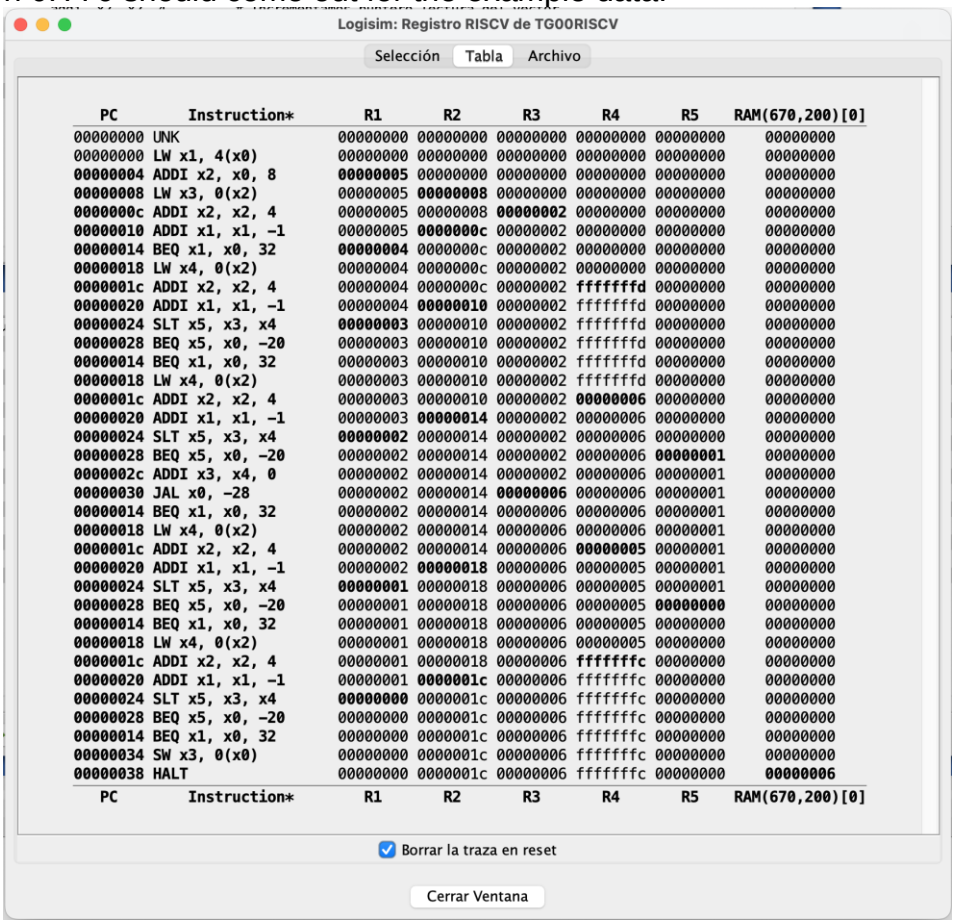
4. Adapt the addresses and header

Start by inserting the header `v2.0 raw` in the files. Delete lines with content `00000000` after the last element of the vector `fffffffc`, remember -4 in the example.

In the instructions *instmem.txt*, we have to add the instruction halt, e.g. `0000007f`

5. Simulate the program with the data

Create a trace in Logisim with the **PC, instruction, R1..R5** and data memory position 0. A 6 should come out for the example data.



PC	Instruction*	R1	R2	R3	R4	R5	RAM(670,200)[0]
00000000	UNK	00000000	00000000	00000000	00000000	00000000	00000000
00000000	LW x1, 4(x0)	00000000	00000000	00000000	00000000	00000000	00000000
00000004	ADDI x2, x0, 8	**00000005**	00000000	00000000	00000000	00000000	00000000
00000008	LW x3, 0(x2)	00000005	**00000008**	00000000	00000000	00000000	00000000
0000000c	ADDI x2, x2, 4	00000005	00000008	**00000002**	00000000	00000000	00000000
00000010	ADDI x1, x1, -1	00000005	**0000000c**	00000002	00000000	00000000	00000000
00000014	BEQ x1, x0, 32	**00000004**	0000000c	00000002	00000000	00000000	00000000
00000018	LW x4, 0(x2)	00000004	0000000c	00000002	00000000	00000000	00000000
0000001c	ADDI x2, x2, 4	00000004	0000000c	00000002	**fffffffd**	00000000	00000000
00000020	ADDI x1, x1, -1	00000004	**00000010**	00000002	fffffffd	00000000	00000000
00000024	SLT x5, x3, x4	**00000003**	00000010	00000002	fffffffd	00000000	00000000
00000028	BEQ x5, x0, -20	00000003	00000010	00000002	fffffffd	00000000	00000000
00000014	BEQ x1, x0, 32	00000003	00000010	00000002	fffffffd	00000000	00000000
00000018	LW x4, 0(x2)	00000003	00000010	00000002	fffffffd	00000000	00000000
0000001c	ADDI x2, x2, 4	00000003	00000010	00000002	**00000006**	00000000	00000000
00000020	ADDI x1, x1, -1	00000003	**00000014**	00000002	00000006	00000000	00000000
00000024	SLT x5, x3, x4	**00000002**	00000014	00000002	00000006	00000000	00000000
00000028	BEQ x5, x0, -20	00000002	00000014	00000002	00000006	**00000001**	00000000
0000002c	ADDI x3, x4, 0	00000002	00000014	00000002	00000006	00000001	00000000
00000030	JAL x0, -28	00000002	00000014	**00000006**	00000006	00000001	00000000
00000014	BEQ x1, x0, 32	00000002	00000014	00000006	00000006	00000001	00000000
00000018	LW x4, 0(x2)	00000002	00000014	00000006	00000006	00000001	00000000
0000001c	ADDI x2, x2, 4	00000002	00000014	00000006	**00000005**	00000001	00000000
00000020	ADDI x1, x1, -1	00000002	**00000018**	00000006	00000005	00000001	00000000
00000024	SLT x5, x3, x4	**00000001**	00000018	00000006	00000005	00000001	00000000
00000028	BEQ x5, x0, -20	00000001	00000018	00000006	00000005	**00000000**	00000000
00000014	BEQ x1, x0, 32	00000001	00000018	00000006	00000005	00000000	00000000
00000018	LW x4, 0(x2)	00000001	00000018	00000006	00000005	00000000	00000000
0000001c	ADDI x2, x2, 4	00000001	00000018	00000006	**fffffffc**	00000000	00000000
00000020	ADDI x1, x1, -1	00000001	**0000001c**	00000006	fffffffc	00000000	00000000
00000024	SLT x5, x3, x4	**00000000**	0000001c	00000006	fffffffc	00000000	00000000
00000028	BEQ x5, x0, -20	00000000	0000001c	00000006	fffffffc	00000000	00000000
00000014	BEQ x1, x0, 32	00000000	0000001c	00000006	fffffffc	00000000	00000000
00000034	SW x3, 0(x0)	00000000	0000001c	00000006	fffffffc	00000000	00000000
00000038	HALT	00000000	0000001c	00000006	fffffffc	00000000	**00000006**
PC	Instruction*	R1	R2	R3	R4	R5	RAM(670,200)[0]

Exercise to hand in (now it gets more difficult)

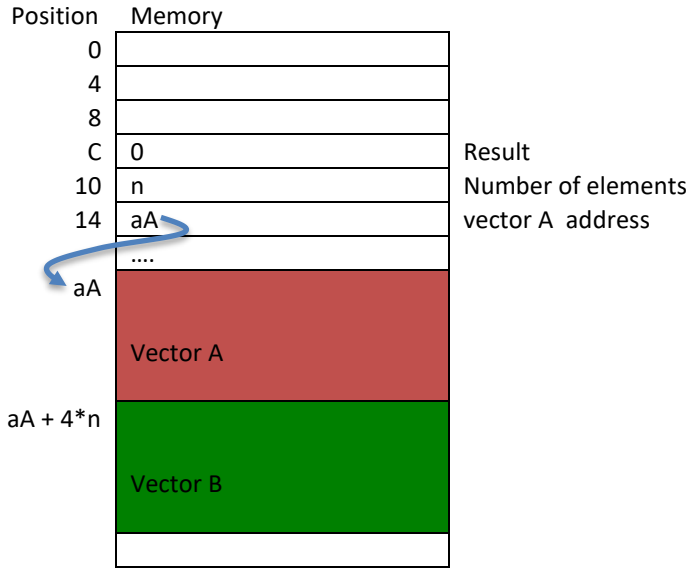
We are going to create a new code based on what we did for calculating the SAD (sum of absolute differences) of two vectors. The description is a bit a hassle, as it comes from old testers. The program to develop should compute the sum of absolute difference (SAD) between two vectors A y B.

$$SAD(A,B) = \sum_i |A[i] - B[i]|$$

The difficulty is the organization of the data in memory:

- The number of elements should be in memory position $10_{(16}$.
- The memory position where A starts should be $14_{(16}$.
- Vector B starts in a position just after the last element of vector A.
- The result should be saved in position $C_{(16}$.
- The first memory positions $0_{(16}$, $4_{(16}$, y $8_{(16}$ can be used freely to store useful variables or data of your program.

The organization of the data memory is therefore:



Get your program into instmem.txt starting from position 0.

You can use RARS as sketched before. The challenge is the organization of the memory. An example of the two vectors with 4 elements (result should be 8) is

```
.data
.word 0, 0, 0            //free space for your use
.word 0                  //result
.word 4                  //n - number of elements of A and B
.word 24                 //address vector A
.word 25, -3, -6, 1      //data vector A
.word 23, -1, -4, -1     //data vector B

.text
//your code
```

If you get SAD working and your simulation table (trace) is similar as the output of RARS, you can upload it into task 5 in GUAC.

Upload the processor together with the memory files "*instmem.txt*" and "*datamem.txt*". What guac will do is to put other data, check the result is correct and also measure the time that your program together with circuit took. The faster, the higher the mark.

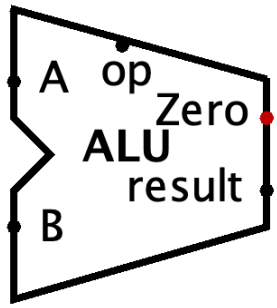
I hope you notice, that your program should also work for the following instance where the result should be 10.

```
.data
.word 0, 0, 0            //free space for your use
.word 0                  //result
.word 5                  //n – number of elements of A and B
.word 28                 //address vector A
.word 0
.word 23, -4, 6, 1, -1   //data vector A
.word 20, -2, 4, -1, 0   //data vector B

.text
//your code
```

Appendix I Implemented components

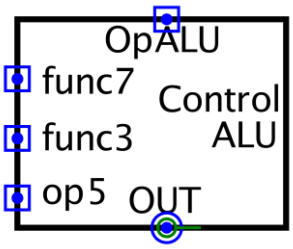
ALU Delay 4 time units (ticks)



Seignals	
A[31:0]	Input data
B[31:0]	Input data
op[3:0]	Operation to carry out by the ALU
result[31:0]	Output data
Zero	Asserted (1) if the result is zero

op[3:0]	ALU operation	
0000	result = A & B	And (bitwise)
0001	result = A | B	Or (bitwise)
0010	result = A + B	add
0110	result = A – B	subtract
0111	If (A<B) result = 1 else result = 0	1 (32 bits) if A is smaller than B
1100	result = A | B	Nor (bitwise)

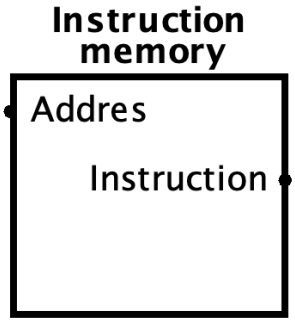
ALU Control Delay 0 time units (ticks)



Signals	
OpALU[1:0]	Operation selection
func7[6:0] func3[2:0] op5	Function codes for ALU/ALUI and bit 5 of the opcode (op)
OUT[3:0]	Code of the ALU operation to execute

OpALU[1:0]	OUT
00	0010 (ALU add)
01	0110 (ALU subtract)
10	Operation determined by func3 and func7 (ALU and ALUI)

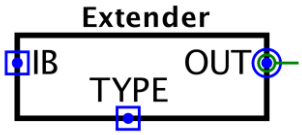
Instruction memory Delay: 4 time units (ticks)



Signals	
Instruction[31:0]	Output bus (reading)
Address[31:0]	Address bus

Memory operation
Continuously: Instruction[31:0] = M[Address[31:0]]

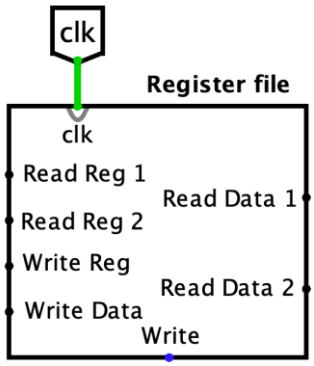
Extender Delay 0 time units (ticks)



Signals	
IB[31:0]	Instruction with an immediate to extend
TYPE[2:0]	Extension type
OUT[3:0]	Immediate extended to 32 bits

TYPE[2:0]	OUT
000	Extend **type-I**: OUT[31:0] = ExtSigno(IB[31:20])
001	Extend **type-S**: OUT[31:0] = ExtSigno(IB[31:25],IB[11:7])
010	Extend **type-B**: OUT[31:0] = ExtSigno(IB[31],IB[7],IB30:25],IB[11:8],0)
011	Extend **type-U**: OUT[31:0] = IB[31:12],000000000000
100	Extend **type-J**: OUT[31:0] = ExtSigno(IB[31],IB[19:12],IB[20],IB[30:21],0)

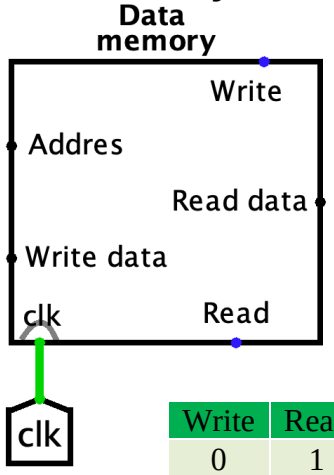
Register file Delay 2 ticks reading, 2 ticks writing



signals	
clk	Clock tick to synchronize writing
Write	Write signal (1)
Read Data 1[31:0]	data read from address in “Read Reg 1”
Read Data 2[31:0]	data read from address in “Read Reg 2”
Write Data[31:0]	Data to write in address in “Write Reg”
Read Reg 1[5:0]	Register address for Reading 1
Read Reg 2[5:0]	Register address for Reading 2
Write Reg[5:0]	Register address for writing

Write	Register operation
0	Reading: Read Data 1[31:0] = Content stored into register of address Read Reg 1[5:0] Read Data 2[31:0] = Content stored into register of address Read Reg 2[5:0]
1	Write on rising CLK edge: Register with address Write Reg[5:0] is loaded with Write Data[31:0] The two reads are also done independent of the Write signal

Data Memory 32 Delay 4 time units (ticks)

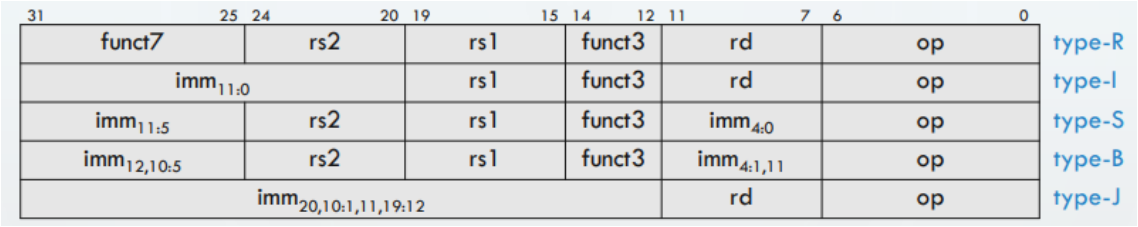


Signal	
clk	Clock tick to synchronize writing
Write	Write signal (1)
Read	read signal (1)
Read data[31:0]	Data bus output (read)
Write data[31:0]	Data bus input (writing)
Addres[31:0]	Address bus

Write	Read	Memory operation
0	1	Read: Read data[31:0] = M(Addres[31:0])
1	0	Write at rising clk edge: M(Addres[31:0]) ← Read data[31:0]

Appendix II Instructions

Instruction format

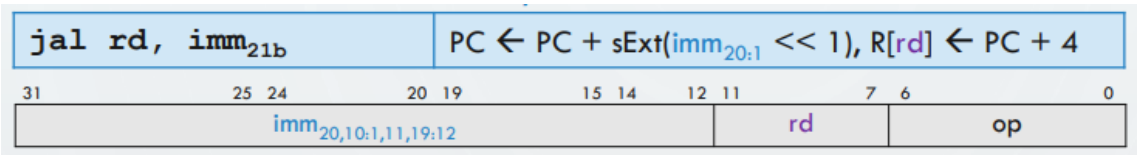


Instructions

The mínimum size instruction set for the processor:

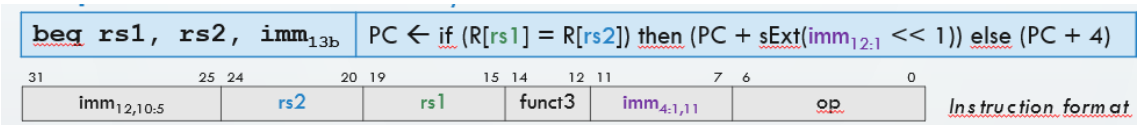
Branching instructions:

Unconditional jump **JAL imm**



Type-J

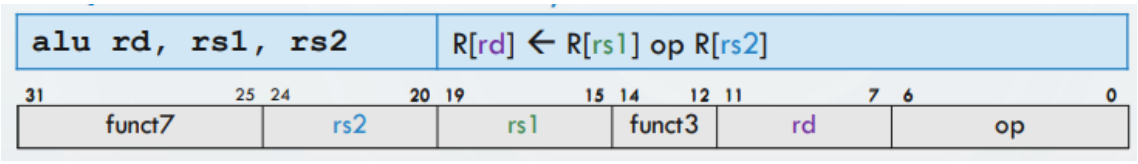
Conditional jump **BEQ rs1, rs2, imm**



Type-B

Transformation instructions

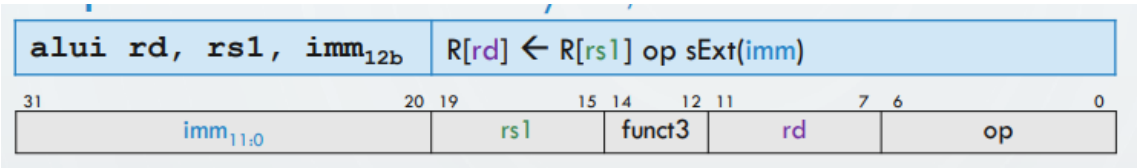
ALU operations **ALU rd, rs, rt**



Type-R

ALU	opALU	funct7	funct3
ADD	+	0000000	000
SUB	-	0100000	000
AND	&	0000000	111
OR	|	0000000	110
SLT	slt	0000000	010

Immediate ALU operations **ALUI rt, rs, imm**

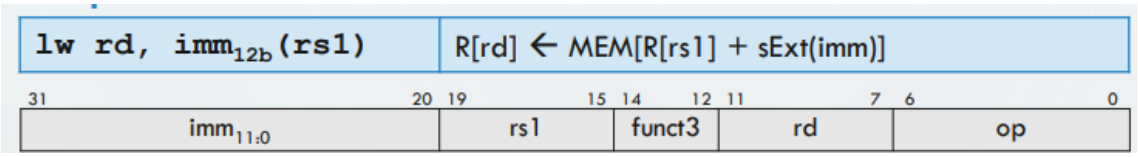


Type-I

ALU	opALU	funct3
ADDI	+	000
ANDI	&	111
ORI	|	110
SLTI	slt	010

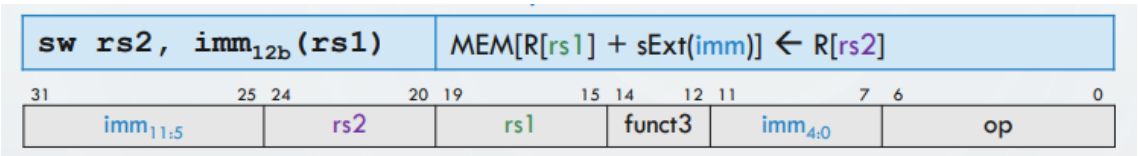
Transference operations

Load **LW rt, desp.(rs)**



Type-I

Store **SW rt, desp.(rs)**



Type-S

Special instruction, just for us

HALT

